

06 · Hardware-Beschleunigung (DMA)

Mikrocomputertechnik 3 · Direct Memory Access — die CPU vom Daten-Schaufeln befreien

Prof. Dr.-Ing. Daniel Klünder

Folie

1 / 75

Willkommen zu Block 6!

- Willkommen zur sechsten Vorlesung in Mikrocomputertechnik 3
- Bisher: Denken (Speicher/Caches), Reagieren (Interrupts), Handeln (PWM), Sprechen (Busse)
- Heute: Effizienz — den größten Stau im Datenpfad auflösen
- Ziel: stupide Fleißarbeit von der CPU auf Hardware-Assistenten verlagern

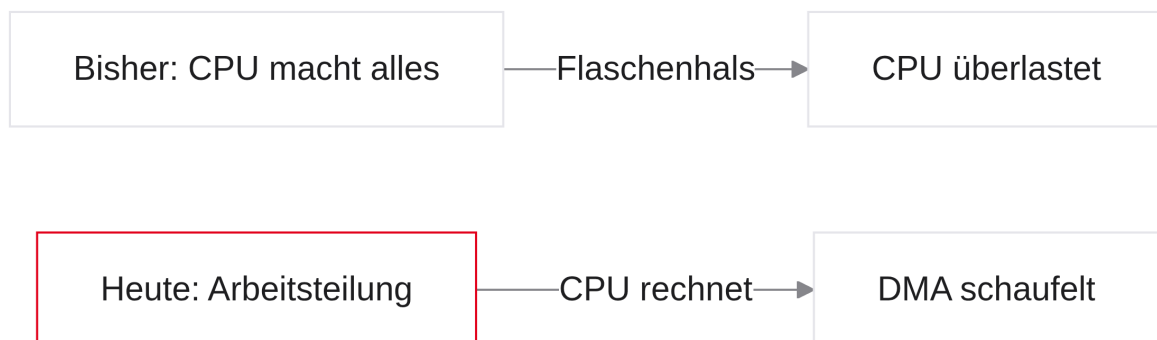
Notizen

Herzlich willkommen zurück! Sensoren und Aktoren beherrschen Sie bereits. Bisher erledigt die CPU jeden Schritt selbst. Heute lernen wir zu delegieren: Die CPU wird Manager, Subsysteme erledigen das Daten-Schaufeln.

Folie

2 / 75

Einordnung im Semester



Notizen

Von „alles selbst“ zu Parallelität: CPU rechnet, DMA bewegt Daten.

Anknüpfung: I2C-Labor

Kurz-Recap aus Block 5 (Vertiefung im Übungsblatt):

- IMU per I2C: Start, Adresse, R/W, ACK/NACK, Stop
- High/Low-Byte per Shift zu 16-Bit-Werten
- CPU wartete oft auf die I2C-Hardware (`wait_for_ack` / Polling)

```
/* Block 5: zusammenhängender Burst, nicht ein einzelnes X-High-  
Polling auf ACK ist bei seltenen Samples tragbar. */  
int read_accel_xyz(uint8_t out[6]);
```

Notizen

Für seltene Sensorwerte ist Polling akzeptabel. Bei großen, kontinuierlichen Datenmengen wird Wartezeit zum Problem.

Schnelle Schnittstellen, langsames Füttern

- Big 3 beherrscht: UART, SPI, I2C
- Externe Geräte anschließbar
- SPI für Displays oft mit SCK im Zehner-MHz-Bereich
- Ein Bit je Takt: 10 MHz ergeben höchstens 1,25 MB/s, 50 MHz höchstens 6,25 MB/s
- DMA verkürzt Nachfüllpausen. Die Leitung wird dadurch nicht schneller

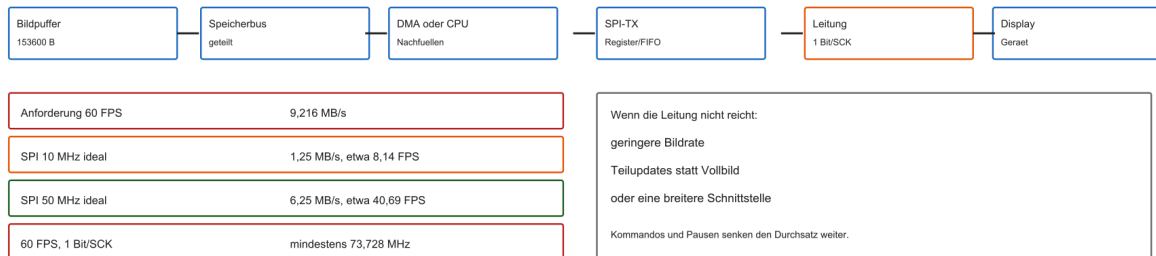
Notizen

Schnelle Peripherie nützt wenig, wenn der Pfad RAM → Controller der Engpass ist.

Das Problem: Datenmenge

Engpass liegt auf der SPI-Leitung, nicht nur in der CPU.

Ein Frame RGB565: $320 \times 240 \times 2 = 153600$ Byte. 60 FPS: $9,216 \text{ MB/s} = 73,728 \text{ Mbit/s}$.



Audio getrennt: 44100 Samples/s mono, 16 Bit = 88200 Byte/s. Samples allein sind keine Byte-Rate.

DMA kuerzt Nachfuehlpausen. Es hebt das SCK-Limit nicht auf. MB hier dezimal.

Abbildung 1: 9,216 MB/s gegen 1,25 und 6,25 MB/s. DMA hebt das SCK-Limit nicht auf.

- Temperaturfühler: wenige Bytes/s — unkritisch
- Audio braucht Kanalzahl und Samplebreite: mono, 16 Bit, 44100 Samples/s sind 88200 Byte/s
- TFT 320×240 , RGB565, 60 FPS:

$$320 \times 240 \times 2 \times 60 = 9\,216\,000 \text{ Bytes/s}$$

- CPU müsste millionenfach RAM lesen und SPI-Register füllen

Notizen

153600 Byte je Frame. Ideal etwa 8,14 FPS bei 10 MHz und 40,69 FPS bei 50 MHz. Für 60 FPS wären mindestens 73,728 MHz SCK nötig, wenn das Gerät das überhaupt erlaubt.

Refresher: Wie die CPU kopiert

- MCT1: Speicherzugriff über Load/Store
- `lw`: RAM → Register; `sw`: Register → MMIO (z. B. SPI-TX)
- Kein „direkt von RAM nach SPI“ in einem Befehl

```
loop:
    lw    t0, 0(s1)      /* RAM lesen */
    sw    t0, 0(s2)      /* Peripherie schreiben */
    addi  s1, s1, 4
    bne   s1, s3, loop
```

Notizen

Die Schleife bewegt Wörter zu vier Byte. „Jedes Byte“ wäre ein anderes Modell mit erlaubtem Byte-Zugriff und TX-Bereitschaft.

Agenda

1. CPU-Flaschenhals: Polling und Interrupt-Stürme
2. DMA: Bypass der CPU und Bus-Mastering
3. Kanäle und Konfiguration (Source, Destination, Increment)
4. Burst, Ringpuffer, Double Buffering
5. Labor-Ausblick (Renode)

Notizen

Zuerst zeigen wir, warum Software scheitert — dann der Hardware-Rettungsanker DMA.

Lernziele Block 6

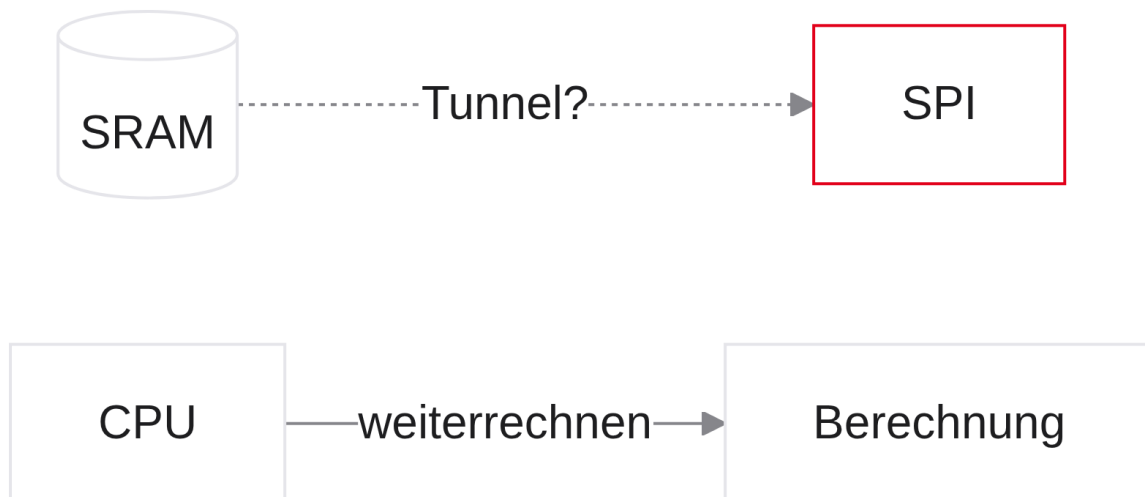
- Begründen, warum Polling und häufige ISRs den Prozessor auslasten
- Bus-Mastering und Arbitration im SoC erklären
- DMA-Kanal: Source, Destination, Increment Mode
- Cache-Kohärenz bei DMA einordnen
- Ringpuffer und Double Buffering (Ping-Pong) für Streams erklären

Notizen

Am Ende: große Transfers nicht mehr per `for`-Schleife über den Bus — sondern per DMA.

Die Leitfrage des Tages

- 1 MB Array im SRAM (z. B. Bild) → SPI → Display
- Parallel: CPU führt schwere Berechnung aus (z. B. Edge-AI)
- Wie fließen die Bytes RAM → Peripherie, ohne die CPU-Rechnung zu stoppen?



Notizen

Ziel: Parallelität — CPU rechnet, Datenfluss läuft daneben. Spoiler: DMA.

Aufgabe — 60 FPS

Warum liefert DMA bei 50-MHz-SPI mit einem Bit je Takt kein RGB565-Vollbild mit 60 FPS?

Notizen

Lösung: 50 MHz tragen ideal 6,25 MB/s, das Vollbild verlangt 9,216 MB/s. Siehe Bandbreitenbild. DMA entfernt Nachfüllpausen, nicht das SCK-Limit.

Methode 1: Polling

```
void send_bild_polling(uint8_t *bild, uint32_t length) {  
    for (uint32_t i = 0; i < length; i++) {  
        while ((SPI_STATUS_REG & TX_EMPTY_FLAG) == 0) { }  
        SPI_DATA_REG = bild[i];  
    }  
}
```

- Warten, bis TX leer — dann nächstes Byte
- Funktioniert korrekt — blockiert die CPU

Notizen

Klassischer Anfänger-Code: zuverlässig, aber die CPU dreht in der Warteschleife.

Die Polling-Falle

- Beispiel: SPI mit 10 MHz-SCK → grob $\approx 0,8 \mu\text{s}$ pro Byte (8 Bit + Overhead je nach Mode)
- 10^6 Bytes $\Rightarrow$ Größenordnung $\approx 0,8$ s Transferzeit
- Die aufrufende Schleife bleibt beschäftigt. Aktivierte Interrupts können weiterlaufen
- 10^6 Byte bei 10 MHz: mindestens 0,8 s reine Leitungszeit, ohne Zusatzpausen

Notizen

Sekundenbruchteile sind für GHz-CPU's Ewigkeiten — Milliarden verschwendeter Takte als „Postbote“.

Methode 2: Interrupt-Versand

- SPI löst bei TX-leer Interrupt aus (über PLIC)
- ISR schreibt ein Byte; CPU rechnet dazwischen weiter

```
volatile uint32_t bytes_sent;

void spi_tx_empty_isr(void) {
    if (bytes_sent < 1000000u) {
        SPI_DATA_REG = bild_array[bytes_sent++];
    } else {
        disable_tx_empty_irq(); /* sonst feuert die Quelle weiter
    }
    /* bytes_sent: übergeben, nicht nachweislich auf der Leitung
}
```

Notizen

Interrupts entlasten gegenüber Polling — bei Mikrosekunden-Raten droht trotzdem Kollaps.

Interrupt-Overhead

Pro Interrupt typisch:

- Kontext sichern. Ein universelles Leeren der Pipeline gibt es nicht
- `mepc/mcause` setzen
- Register retten (Stack)
- Nutzarbeit: ein Byte kopieren
- Register wiederherstellen, `mret`

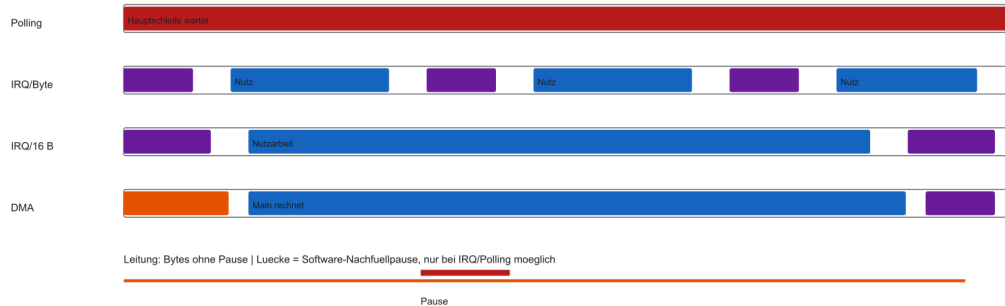
Oft Dutzende Takte Overhead für wenige Nutz-Takte.

Notizen

Erinnerung Block 3: Kontextwechsel ist teuer — besonders bei einem Byte Nutzlast.

Interrupt Storm

Dieselbe Leitung, vier CPU-Spuren. 10 MHz SPI: ein Byte ideal 0,8 μ s.
1 000 000 Byte mindestens 0,8 s reine Leitungszeit. Ausschnitt, nicht massstaeblch.



Modell: 100-MHz-CPU, 100 Zyklen je Byte-IRQ. 1,25 Mio IRQ/s waeren 125 Prozent. Nicht erreichbar.

16 Byte je IRQ: derselbe feste Aufwand etwa 7,81 Prozent. Kopieren kommt dazu. Keine Messung.

Polling beschaeftigt die aufrufende Schleife. Aktivierte Interrupts koennen weiterlaufen.

Abbildung 2: Vier CPU-Spuren, dieselbe Leitung. 100 Zyklen je Byte-IRQ reichen bei 10 MHz nicht.

- SPI meldet „leer“ im Sub- μ s- bis μ s-Takt
- Millionen ISRs/s moeglich
- CPU: Zeit geht in Push/Pop statt Nutzarbeit

Phase	Was passiert
kurz	KI/Main
lang	ISR-Overhead
kurz	KI/Main
lang	ISR-Overhead

Notizen

Interrupts passen zu seltenen Ereignissen (Taste, Tick) — nicht zu Dauerfeuer im Mikrosekunden-Takt.

CPU als Postbote

- Polling und ISR: jedes Byte durch CPU-Register
- Pfad: RAM → Bus → (Cache) → Register → Bus → SPI
- ALU/Pipeline involviert, obwohl nur 1:1 kopiert wird

Notizen

Hochkomplexe Rechenwerke als Spediteur missbrauchen — architektonisch unnötig.

Bus-Engstelle beim Software-Kopieren

- Auch mit getrennten L1-I/D-Caches teilen sich CPU und Peripherie oft den Systembus zum Hauptspeicher
- Intensives lw/sw -Kopieren belegt diesen Pfad
- Instruction-Fetch und Nutzarbeit konkurrieren → Pipeline stottert

Notizen

Klassisch als Von-Neumann-Flaschenhals diskutiert; bei modernen SoCs oft die gemeinsame Memory-Interconnect. Software-Kopieren belegt denselben teuren Pfad.

Lösung: Hardware-Assistent

- CPU = Manager (rechnen, entscheiden)
- Auftrag: Quelle, Ziel, Länge — dann weiterarbeiten
- Assistent schaufelt und meldet „fertig“
- Name: Direct Memory Access (DMA) / DMAC

Notizen

Einmal konfigurieren, Hintergrundtransfer, Interrupt erst am Ende (oder bei Meilensteinen).

Fragen? — Flaschenhals

- Warum Interrupt-Sturm bei schnellem SPI?
- Warum verschwenden $\downarrow w / \uparrow sw$ -Schleifen ALU/Bus?
- „Postboten“-Problem klar?

Notizen

Als Nächstes: Was DMA ist und wie Bus-Mastering funktioniert.

Was ist DMA?

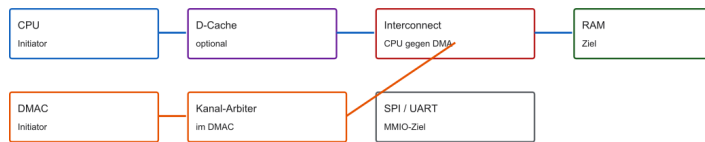
- DMA: Direct Memory Access — direkter Speicherzugriff
- Peripherie (SPI, UART, ADC, ...) liest/schreibt RAM ohne CPU-Kopierschleife
- Steuerung: DMA-Controller (DMAC) als dediziertes Subsystem

Notizen

Geräte umgehen die CPU und greifen — über den DMAC — direkt auf den Speicher zu.

Bypass der CPU

CPU und DMAC sind Initiatoren. RAM und Peripherieregister sind Ziele.



DMAMUX routet Requests. Kanäle sind Kontexte, nicht automatisch eigene Datenpfade.



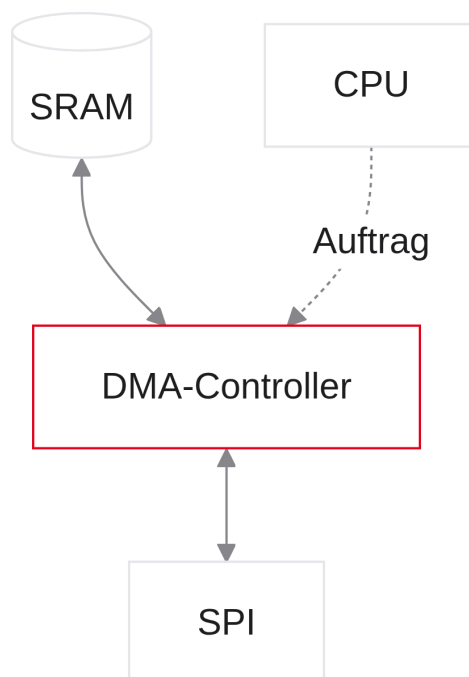
Round-Robin, angenommen: CPU, DMA, CPU, DMA. Feste Priorität: DMA, DMA, CPU.

Nicht: DMA gewinnt immer. Nicht: Stalls sind immer kurz.

Setup, Cache-Wartung und Abschluss kosten Zeit. Bei kleinen Kopien kann die CPU günstiger sein.

Abbildung 3: CPU und DMAC sind Initiatoren. Zwei Arbitr: im DMAC und am Interconnect.

- Ohne DMA: RAM → CPU → UART/SPI
- Mit DMA: RAM → DMAC → Peripherie
- Der Kopierpfad läuft am Registerfile vorbei. Setup, Cache und Abschluss bleiben
- Buskonflikte können die CPU trotzdem verzögern



Notizen

CPU erteilt den Auftrag und rechnet weiter; der Tunnel RAM–Peripherie läuft autonom.

Der DMA-Controller (DMAC)

- Eigenes Silizium-Modul auf dem SoC
- Keine allgemeine ALU — spezialisiert auf Adressen und Transfers
- Merkt sich u. a. Quelle, Ziel, Restlänge
- Oft mehrere Controller/Kanäle (DMA1, DMA2, ...)

Notizen

Lesen, schreiben, Zeiger, Zähler. Bei kleinen Längen können Setup und Cache-Wartung die CPU-Schleife teurer machen als der Transfer selbst.

Kampf um den Bus

- Oft ein gemeinsamer Pfad zum RAM
- Früher: CPU alleiniger Nutzer
- DMA will denselben Bus — Konfliktregelung nötig

Notizen

Zwei Master am gleichen Medium brauchen Arbitration.

Bus-Mastering

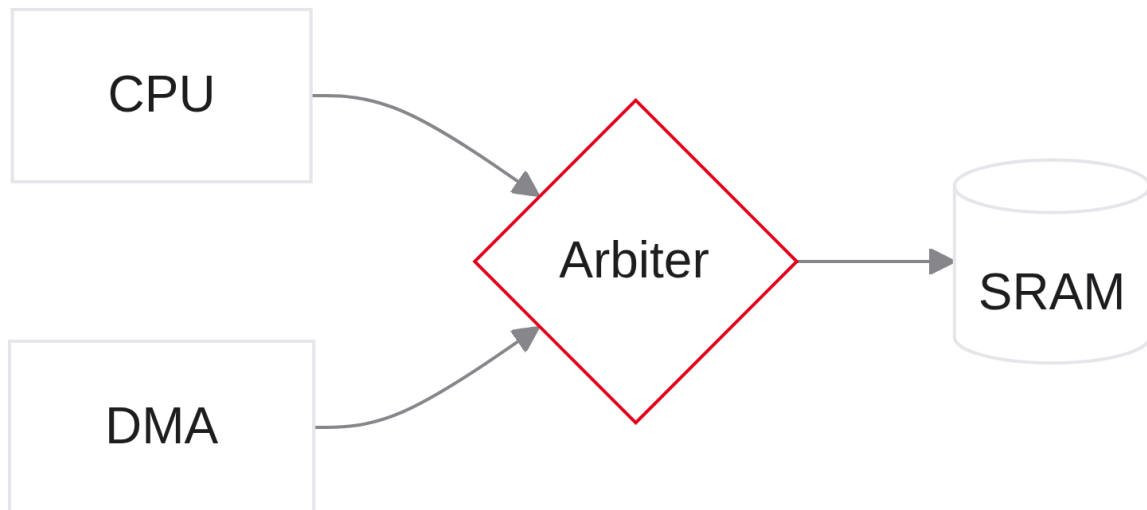
- Bus-Master: darf selbst Transaktionen starten
- CPU: Bus-Master
- DMAC: ebenfalls Bus-Master
- UART/SPI-Datenregister typisch: Slave (passiv)

Notizen

Die CPU verliert das Monopol; der DMAC darf eigenständig Speicherzugriffe auslösen.

Bus-Arbitration

- Bus-Matrix / Arbiter steuert den Zugang
- Gleichzeitiger Zugriff CPU und DMA → Priorität entscheidet
- Priorität ist eine Politik, kein Naturgesetz. Round-Robin und feste Priorität unterscheiden
- „DMA gewinnt immer“ und „Stalls sind immer kurz“ gelten nicht allgemein



Notizen

Echtzeit-Streams vertragen Stalls schlecht; die CPU verkraftet kurze Wartezeiten besser.

Cycle Stealing

- DMA „klaut“ gelegentlich Bus-Zyklen
- Kein ISR-Overhead pro Byte → Störung oft gering
- Mit L1-Cache rechnet die CPU oft ohne Dauerzugriff auf den Hauptspeicherbus

Notizen

Cache-Hits vertragen Cycle Stealing besser als eine CPU, die selbst ständig den RAM braucht.

Transparent vs. Blocking

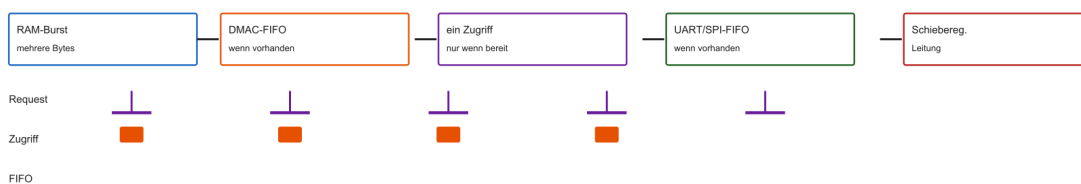
- Transparent: DMA nutzt Bus, wenn die CPU ihn ohnehin nicht braucht
- Blocking / Burst: DMA belegt den Bus länger — CPU stottert kurz
- Immer noch weit günstiger als C-Kopierschleifen

Notizen

Ein Burst kann günstiger sein als eine Kopierschleife. Er ist nicht in jedem Fall billiger: Setup, Länge und Buslast entscheiden.

Hardware-Handshake

Speicher-Burst und Peripherie-Transfer sind zwei Breiten. DREQ/DACK sind Lehnnamen.



Ein Request erlaubt nicht 16 Words in ein Ein-Byte-TX-Register.

M2P: RAM -> Peripherie, P2M: Peripherie -> RAM, M2M: RAM -> RAM.

P2M verhindert einen Overrun nicht, wenn der Pfad langsamer ist als die Quelle.

M2M ist kein allgemeines memmove fuer Ueberlappung. DACK ist kein I2C-ACK.

Die zwei FIFOs nur zeichnen, wenn das konkrete Modell sie hat.

Abbildung 4: Speicher-Burst und ein peripheriegesteuerter Zugriff haben verschiedene Breiten.

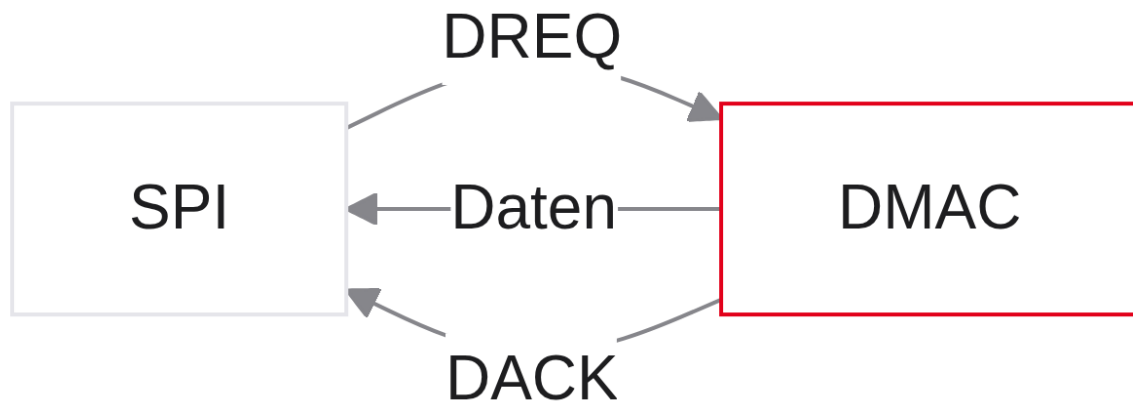
- DMA darf Peripherie nicht überfluten
- Synchronisation über dedizierte Signale im SoC (ohne CPU)

Notizen

Wie Staffelübergabe: erst anfordern, dann liefern.

Request / Acknowledge (Handshake)

- Request (klassisch oft DREQ): Peripherie „Register leer / bereit“
- DMAC: Transfer RAM → Peripherie
- Acknowledge (klassisch oft DACK): „geliefert“
- Signale und Namen sind herstellerspezifisch — das Prinzip bleibt gleich

**Notizen**

Klassische Bezeichnungen DREQ/DACK; am Chip oft andere Signalnamen. Prinzip: Peripherie fordert an, DMA liefert.

Richtung: Memory → Peripheral

- SRAM → UART / SPI / I2C / DAC
- Quelle: Array (Adresse steigt); Ziel: festes Datenregister
- Typisch: Audio-Out, Display-Refresh

Notizen

Klassischer Sender: C-Puffer hinaus in die Welt.

Richtung: Peripheral → Memory

- UART / SPI / ADC → SRAM
- z. B. ADC sampelt Mikrofon mit hoher Rate → DREQ → DMA füllt Puffer
- CPU wertet später aus (Spracherkennung etc.)

Notizen

Sensorik ohne Interrupt-Sturm: DMA füllt den Puffer geräuschlos.

Richtung: Memory → Memory

- SRAM-Array A → Array B
- Ersetzt/beschleunigt `memcpy`-artige Kopien
- Oft ohne externes DREQ — so schnell der Bus erlaubt

Notizen

Auch reine Software-Pufferumzüge profitieren vom DMAC.

DMA-Interrupts

Besitz des Puffers und Abschluss der Leitung sind zwei Zeitachsen.



Während DMA besitzt: CPU ändert den aktiven Quellbereich nicht.

Ein lokales Array darf nach Rückkehr der Startfunktion nicht verschwinden.



Abstand nicht festgelegt. Ohne Controllerdokument keine feste Differenz.

Pufferfreigabe, nächster DMA-Start und SPI-CS sind getrennte Entscheidungen.

Vor Start: Kanal stoppen, Altstatus, Route, Breite, Count, Cache, IRQ. Kein blindes |=.

Abbildung 5: DMA-TC und das leere Schieberegister sind zwei Zeitpunkte.

- Anbindung an PLIC / Interrupt-System
- Typisch: Transfer Complete (TC), Half Transfer (HT), Transfer Error (TE)
- Ein Interrupt nach N Bytes statt N Interrupts

Notizen

HT ist Basis für Ping-Pong: eine Hälfte füllen/lesen, während die andere transferiert.

Fragen? — DMA-Grundlagen

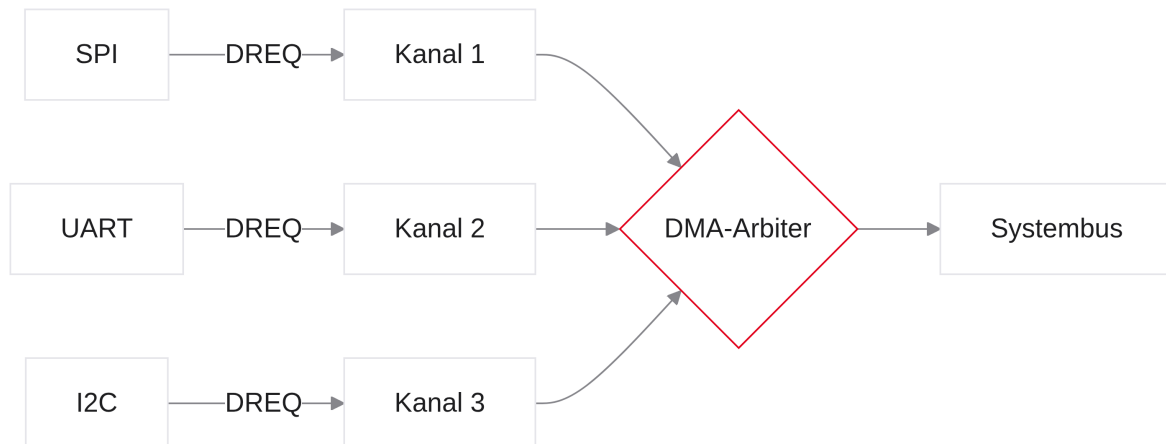
- Was bedeutet Bus-Mastering?
- Wie schlichtet der Arbiter (Cycle Stealing)?
- Rolle des Hardware-Handshakes (Request/Acknowledge)?

Notizen

Als Nächstes: Kanäle konfigurieren — Source, Destination, Increment, Cache.

DMA-Kanäle (Channels / Streams)

- Viele Peripherien wollen parallel Daten bewegen (UART, SPI, I2C, ...)
- Ein DMAC hat daher mehrere Kanäle (z. B. Channel 0–7)
- Jeder Kanal: eigene Quelle, Ziel, Länge, Priorität
- Interner DMA-Arbiter: bei gleichzeitigen DREQs entscheidet die Kanalpriorität



Notizen

Mehrere Förderbänder parallel: Audio auf Kanal 1, Logs auf Kanal 2 — der DMA-interne Schiedsrichter regelt Konflikte, ohne die CPU.

Multiplexing der Kanäle (DMAMUX)

- Oft mehr Peripherien als DMA-Kanäle
- Lösung: Multiplexer-Matrix (häufig DMAMUX) vor dem DMAC
- Software verbindet z. B. UART_TX-Request mit Kanal 3
- Doppelbelegung desselben Kanals → undokumentiertes / fehlerhaftes Verhalten
- Referenzhandbuch: Request-Tabellen sind Pflichtlektüre

Notizen

Erster Programmierschritt: Request-Leitung der richtigen Peripherie auf den richtigen Kanal legen.

MMIO-Register eines Kanals

- Steuerung über Memory-Mapped Register
- In C oft als `struct` abgebildet
- Vier Kernfragen: Woher? Wohin? Wie viel? Welche Breite / Modi?

```
typedef struct {  
    volatile uint32_t SRC_ADDR;    /* Quelle */  
    volatile uint32_t DST_ADDR;    /* Ziel */  
    volatile uint32_t COUNT;       /* Anzahl */  
    volatile uint32_t CONTROL;     /* Breite, Enable, ... */  
} DMA_Channel_TypeDef;  
  
/* fiktiv - Labor: Datenblatt / Renode */  
#define DMA_CH1 ((DMA_Channel_TypeDef *)0x40020000u)
```

Notizen

Layout und Bitfelder sind herstellerspezifisch — die vier Parameter finden Sie überall wieder.

Parameter 1: Source Address (SRC)

- Physikalische Startadresse der Quelle in `SRC_ADDR`
- Senden: oft Adresse eines Arrays im SRAM
- DMA liest von dort (ggf. mit Auto-Inkrement)

```
uint8_t bild_array[1000];  
DMA_CH1->SRC_ADDR = (uint32_t)&bild_array[0];
```

Notizen

Lade-Ort: Read-Zugriff auf dem Systembus in den internen DMA-Pfad.

Parameter 2: Destination Address (DST)

- Zieladresse in `DST_ADDR`
- Senden über SPI/UART: Adresse des Datenregisters (z. B. `TX_DATA`)
- DMA schreibt blind auf diese Adresse

```
#define SPI_TX_DATA_REG 0x4003000Cu /* fiktiv */

DMA_CH1->DST_ADDR = SPI_TX_DATA_REG;
```

Notizen

Ablade-Ort: nach dem Lesen folgt der Write auf die Peripherie — der Controller startet den seriellen Versand.

Parameter 3: Transfer Count

M2P, fiktiv: 8 Bit, COUNT 16, SRC ab 0x20001000, DST fest 0x4003000C.

n	SRC	DST	Rest
1	0x20001000	0x4003000C	15 P2M, halfwordfaehig angenommen
2	0x20001001	0x4003000C	14 ADC 0x40031000, fest
3	0x20001002	0x4003000C	13 RAM ab 0x20002000, +2
4	0x20001003	0x4003000C	12 COUNT 8 = 16 Byte

Ein 16-Bit-H2C-Wert kann zwei Byte-Transfers sein. C-Typ und erlaubte Breite sind verschieden.

1 000 000 Byte, 8 Bit, COUNT max. 65535: $15 \times 65535 + 16975 = 16$ Bloেকে.

Adressregister in Bytes zaehlen. `uint16_t* p; p += width` skaliert doppelt.

Getrennt: Bus-Alignment, Cacheline-Alignment, DMA-Erreichbarkeit.

Ziel-Increment auf MMIO kann Nachbarregister beschreiben.

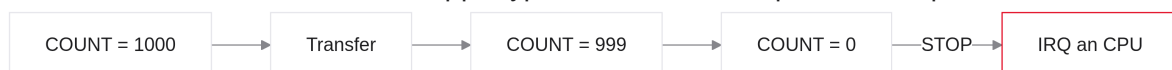
`uint32_t`-Cast ist nur eine Adressannahme. `uintptr_t` macht aus einer CPU-Adresse keine DMA-Adresse.

Alle Adressen in dieser Abbildung sind fiktives Lehrmodell.

Packing nur, wenn Quelle, Ziel und Controller es dokumentieren.

Abbildung 6: Vier fiktive Byte-Transfers. Das UART-Ziel 0x4003000C bleibt fest.

- COUNT zählt Transferelemente, nicht automatisch Bytes
- Beispiel: höchstens 65535 Elemente. 1000000 Byte zu 8 Bit brauchen 16 Blöcke: fünfzehnmal 65535 plus 16975
- Nach erfolgreichem Transfer: Hardware dekrementiert automatisch
- Bei COUNT = 0: Kanal stoppt, typisch Transfer-Complete-Interrupt



Notizen

Ohne Zähler würde der DMA endlos weiterlaufen. Null bedeutet: Auftrag erledigt, CPU benachrichtigen.

Folie

41 / 75

Parameter 4: Data Width

- Pro Transfer: 8, 16 oder 32 Bit — nur wenn Quelle, Ziel und DMA-Pfad das erlauben
- Ein `uint16_t` im C-Programm rechtfertigt keinen Halfword-Zugriff auf ein Byte-Register

C-Typ	DMA-Breite	Transfers für 32 Bit Nutzdaten
<code>uint8_t</code>	Byte (8)	4
<code>uint16_t</code>	Halfword (16)	2
<code>uint32_t</code>	Word (32)	1 (oft maximal effizient)

Notizen

Die erlaubte Breite steht im Controller und im Zielregister. Ein 16-Bit-Messwert über einen Byte-Pfad bleibt zwei Byte-Transfers. Die Tabelle ist nur die Umrechnung, kein Freibrief für Halfword-MMIO.

Folie

42 / 75

Address Increment Mode

- Arrays: Pointer muss wandern — sonst 1000× dieselbe Adresse
- INC: nach jedem Transfer + Data-Width auf SRC und/oder DST
- Hardware-Äquivalent zu `pointer++`
- Quelle und Ziel getrennt schaltbar

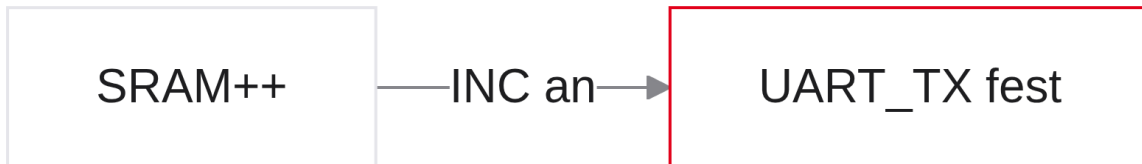
```
/* Konzept - in Hardware als Auto-Inkrement */
*DST_ADDR = *SRC_ADDR;
SRC_ADDR += width; /* wenn Memory-Increment aktiv */
```

Notizen

Damit wird die klassische Kopierschleife in Silizium nachgebaut.

Memory → Peripheral: Pointer

- Source Increment: **an** (Array durchlaufen)
- Destination Increment: **aus** (festes TX-Register)
- Ziel inkrementieren → Schreiben neben das Register → oft Bus Fault

**Notizen**

Klassischer Fehler: Peripherie-Adresse mitwandern lassen. Der Briefschlitz bleibt starr.

Peripheral → Memory: Pointer

- Source Increment: **aus** (festes ADC-/RX-Register)
- Destination Increment: **an** (Werte im Array ablegen)

**Notizen**

Sensor immer aus demselben Fach; Ablage im RAM der Reihe nach.

Memory Alignment

- Word-Transfers (32 Bit): Startadressen typisch 4-Byte-aligned
- Unaligned (z. B. `0x20000001`) → oft Bus Error / Transfer Error
- In C Alignment erzwingen:

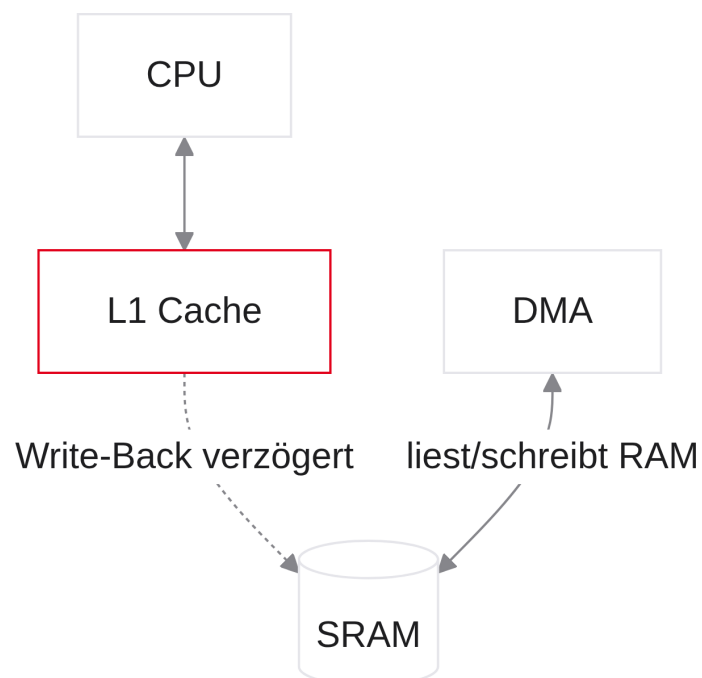
```
__attribute__((aligned(4))) uint8_t dma_buffer[1000];
```

Notizen

Unaligned Word-Zugriffe wären teuer oder verboten — DMA-Puffer sauber ausrichten.

Cache-Kohärenz und DMA

- Relevant, sobald ein Data-Cache (Write-Back) vorhanden ist — viele einfache MCUs haben keinen
- Erinnerung Block 2: L1-D-Cache puffert RAM-Inhalte
- DMA sieht den CPU-D-Cache typischerweise **nicht** — Zugriff auf den physischen Speicher
- CPU und DMA können unterschiedliche Sicht auf dieselben Adressen haben



Notizen

Klassische Falle: CPU ändert Array nur im Cache; DMA verschickt veraltete RAM-Bytes.

Stale Data (beide Richtungen)

Nicht kohärenter Write-Back-D-Cache. Werte beispielhaft. Linie hier 64 Byte, nur illustrativ.



prepare_tx_for_device: neu veroeffentlichen, bevor DMA liest.

prepare_rx_for_device: schmutzige Kopie entfernen, bevor das Geraet schreibt.

finish_rx_for_cpu: veraltete Kopie verwerfen, bevor die CPU liest.

Clean schreibt zurueck. Invalidate verwirft. Flush = Clean plus Invalidate. FENCE ersetzt das nicht.

Auch andere nicht kohärente Modi koennen beim Empfang alte Kopien liefern. Zicbom nur wenn vorhanden.

Abbildung 7: TX veröffentlicht 0x22 vor dem Start. RX darf eine schmutzige Zeile nicht später zurückschreiben.

- Senden (CPU → DMA): frische Daten nur im Cache → DMA sendet alte RAM-Inhalte
- Empfangen (DMA → CPU): frische Daten im RAM → CPU trifft veralteten Cache-Hit
- TX: Daten vor dem DMA-Start veröffentlichen. RX: Bereich vor der Übergabe vorbereiten und nach Abschluss synchronisieren
- Nur „danach invalidieren“ schützt nicht vor einer schmutzigen Zeile, die später zurückschreibt

Notizen

Beide Seiten können sich „belügen“, wenn Kohärenz nicht hergestellt wird.

Cache Clean / Invalidate

- Clean (Flush): vor DMA-Sendestart schmutzige Lines des Puffers in den RAM schreiben
- Invalidate: vor CPU-Lesen empfangener DMA-Daten Lines verwerfen → nächster Read geht zum RAM
- API ist plattformabhängig (ARM: Cache-Maintenance; RISC-V: Fence ordnet Zugriffe; für Cache-Kohärenz ggf. Zicbom — `cbo.clean/cbo.inval`)

```
bild_berechnen();  
/* Konzept: Cache-Bereich in den RAM schreiben */  
cache_clean((uintptr_t) bild_array, sizeof(bild_array));  
dma_start();
```

Notizen

Wenige Takte Overhead — dafür korrekte Sichtbarkeit zwischen CPU und DMA.

Uncached DMA-Zonen

Drei Ebenen. Ein Linkername macht Speicher nicht uncached.



Fiktive Karte: 246 KiB ab 0x20000000, 10 KiB ab 0x2003D800, Ende exklusiv 0x20040000.



Uncached nur, wenn die Plattform das fuer 0x2003D800 wirklich konfiguriert.

PMP	Zugriffsrechte
PMA / Plattform	unter anderem Cacheability
Linker	nur Platzierung

Uncached heisst weder automatisch DMA-erreichbar noch automatisch geordnet.

Abbildung 8: PMP schützt Zugriffe. Cacheability kommt von der Plattform, nicht vom Linkernamen.

- Alternative nur, wenn die Plattform einen nicht gecachten Bereich bereitstellt
- Standard-RISC-V-PMP regelt Zugriffsrechte, nicht Cacheability. Dafür sind PMAs oder die Plattform zuständig
- Der Linker legt `.dma_buffer` in einen Bereich, dessen Attribute schon gelten
- Uncached ist nicht automatisch DMA-erreichbar und nicht die einzige professionelle Lösung

```

MEMORY {
    RAM_CACHED      : ORIGIN = 0x20000000, LENGTH = 246K
    RAM_UNCACHED    : ORIGIN = 0x2003D800, LENGTH = 10K /* DMA */
}
  
```

Notizen

Kohärentes DMA, explizite Wartung und uncached Puffer sind drei systemabhängige Wege. Der Abschnittsname `RAM_UNCACHED` ändert die Hardware nicht.

Aufgabe — Adresse und Linker

Welche Adresse bleibt beim UART-TX fest? Macht der Name `RAM_UNCACHED` den Bereich uncached?

Notizen

Das TX-Register bleibt fest, die Quelladresse steigt. Cacheability setzt die Plattform. Der Linker platziert nur. Siehe Adressspur und die drei Ebenen.

Single-Transfer vs. Burst

- Single: ein DREQ → typisch ein Read + ein Write; Bus danach wieder frei
- Burst: nach einem Request ein Block (z. B. 16 Words) am Stück
- Burst: höherer Durchsatz, kurz härtere Busbelegung für die CPU

Notizen

Single ist höflich; Burst maximiert Bandbreite, blockiert den Bus länger.

Warum Burst-Transfers?

SoC-Transaktionen und DRAM-Kommandos sind zwei Ebenen.

Bus, angenommen: Einzelbeats brauchen je eine Anfrage. Burst bündelt Beats.



DRAM, Open-Page, dieselbe zunaechst geschlossene Zeile:

ACT Row-Hits auch bei getrennten SoC-Zugriffen

Zeilenwechsel: RD, PRE, ACT, RD. Offene passende Zeile: kein neues ACT.

Nicht: 16 Einzeltransfers bedeuten 16 Row-Aktivierungen.

Vergleichen: Protokollaufwand, räumliche Lokalität, CPU-Wartezeit. Keine feste Beschleunigung.

Längere Busbelegung ist der Preis des Bursts. Arbitration je Beat ist nur eine angenommene Politik.

On-Chip-SRAM hat keine DRAM-Zeile. Der Bezug auf Vorlesung 1 gilt nur fuer DRAM.

Eine DMA-Länge ist nicht automatisch die DRAM-Burstlänge.

Abbildung 9: Einzelzugriffe können dieselbe DRAM-Zeile treffen. 16 Aktivierungen sind keine Regel.

- On-Chip-SRAM: Burst spart vor allem wiederholte Bus-Arbitration
- Externes DRAM (Block 1): Zeile öffnen kostet; offene Zeile → schnelle Column-Zugriffe
- Eine offene Zeile kann mehrere Reads bedienen, auch bei getrennten SoC-Zugriffen
- Ein Zeilenwechsel braucht PRE und ACT. SRAM hat diese Zeilenlogik nicht

Modus	Typischer Nachteil Single	Vorteil Burst
On-Chip-SRAM	16× Arbitration	1× Arbitration, hoher Durchsatz
DRAM	Zeile kann schon offen sein	Row-Hits hängen von Abbildung und Politik ab

Notizen

SRAM-MCU: Burst = weniger Bus-Schiedsverfahren. DRAM-SoC: zusätzlich die Zeilenlogik aus Block 1.

Folie

53 / 75

FIFOs im DMAC

- Problem: Burst holt schnell aus dem RAM; SPI/UART sind langsamer
- Viele DMACs: interner FIFO als Zwischenpuffer
- Burst → FIFO füllen → Bus freigeben → Peripherie per DREQ aus dem FIFO füttern



Notizen

Entkopplung: schneller Speicherzugriff und langsamer I/O-Takt getrennt.

Folie

54 / 75

Problem endloser Streams

- Audio-DAC: z. B. $\approx 44\,100$ Samples/s, lange Dauer
- RAM fasst nicht die ganze Datei
- Kleiner Puffer: `COUNT = 0` → Sound bricht ab
- Ständiges Stoppen/Neustarten in Software → Glitches

Notizen

Feste Blöcke reichen für Streams nicht — der DMA muss „im Kreis“ weiterlaufen können.

Circular Mode (Ringpuffer)

- Array fester Größe als Ring (z. B. 4 KiB)
- Bei `COUNT = 0`: Pointer und Zähler werden in Hardware neu geladen (Wrap)
- DMA rotiert, bis Software den Kanal stoppt

Notizen

Industriestandard für Audio, Video, Netzwerk: Speicher als Endlosschleife.

Ringpuffer: Software-Pflicht

- DMA-Lesezeiger läuft im Kreis
- CPU muss den gerade verlassen Bereich nachfüllen (z. B. aus Flash/Netz)
- Zu langsam → alte Daten → Stottern (Buffer Underrun)

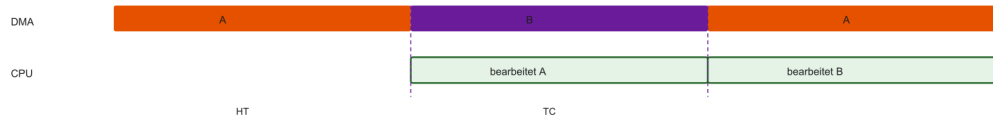


Notizen

Katz-und-Maus: DMA frisst, Software füllt — wo steht der DMA gerade?

Double Buffering (Ping-Pong)

44100 Hz, mono, 16 Bit: 88200 Byte/s. Puffer 4096 B, Haeften 2048 B = 1024 Samples.
16-Bit-COUNT 2048. HT nach 1024 Elementen. Eine Haelfte ideal etwa 23,22 ms, Runde 46,44 ms.



Beide Haeften vor dem Start vorbereiten. Danach nur die freigegebene Haelfte anfassen.

IRQ-Verzoegerung + Nachfuellen + Veroeffentlichen + Reserve < Fenster vor der naechsten Nutzung.

30 ms Nachfuellzeit ueberschreitet die idealen 23,22 ms. TX: Underrun. RX: Overrun.

ISR: Status lesen, Quelle quittieren, Arbeit signalisieren. Nachfuellen ausserhalb, wenn das Budget traegt.

Zwei boolesche Flags zaehlen keine beliebig vielen Runden. Bei unklarem Besitz neu synchronisieren.

Etwa 43,07 HT-/TC-Ereignisse je Sekunde. Wrap nur, wenn der Controller ihn neu laedt.

Linearer Ring mit HT/TC und Hardware-Doppelpuffer sind verschiedene Implementierungen.

Abbildung 10: Eine Haelfte dauert ideal etwa 23,22 ms. 30 ms Nachfuellen verpasst das Fenster.

- Ring in zwei Haeften A und B
- DMA liest A → CPU füllt parallel B (und umgekehrt)
- Kein Konflikt: getrennte Bereiche

Notizen

Zwei Eimer: einer wird geleert, der andere gefüllt — stabile Multimedia-Wiedergabe.

Ping-Pong: Interrupt-Logik

- Mitte erreicht → Signal: Haelfte A fertig → CPU füllt A nach
- Ende erreicht → Signal: Haelfte B fertig → Wrap, CPU füllt B nach
- Endlos mit klarer Phasengrenze

Notizen

Zwei Ereignisse reichen: Halb und Voll (Complete) im Circular Mode.

Half-Transfer-Interrupt (HT)

- Bei vielen Controllern: HT bei exakt 50 % von COUNT → erste Hälfte nachfüllen
- TC (COUNT = 0): zweite Hälfte nachfüllen (bei Circular: Wrap)
- Ein Array + zwei Flags in der ISR (Namen/Bits herstellerspezifisch)

```
volatile unsigned ht_count, tc_count;

void dma_isr(void) {
    if (STATUS_REG & HALF_TRANSFER_FLAG) {
        clear_ht(); /* Semantik laut Controller, bei W1C kein RM
        ht_count++;
    }
    if (STATUS_REG & TRANSFER_COMPLETE_FLAG) {
        clear_tc();
        tc_count++;
    }
}

/* Nachfüllen im Worker, nicht als lange Arbeit in der ISR */
```

Notizen

HT + TC = Double Buffering ohne zwei separate Hardware-Kanäle.

Ausblick: DMA2D (2D-Kopie)

RGB565, 320 Pixel, Pitch 640 Byte. Rechteck 4 x 3 bei x=10, y=20.



Basis 0x20001000, fiktiv

Erste Zeile: 0x20004214

8 Byte kopieren, Skip 632 Byte

Nächster Zeilenstart +640 Byte

Dichtes Ziel: Startabstand 8 Byte

Offsetregister können Pixel statt Bytes zählen. Datenblatt umrechnen.

Scatter-Gather: drei Puffer, Deskriptor mit SRC, DST, Count, Next.

Deskriptoren brauchen Erreichbarkeit, Alignment, Lebensdauer und ggf. Cache-Pflege.

Next = 0 ist kein universeller Stopp. DMA ersetzt kein I2C-START/STOP.

Abbildung 11: Rechteckstart 0x20004214. Acht Byte je Zeile, Quellpitch 640 Byte.

- High-End-SoCs: 2D-DMA / Grafik-Accelerator (z. B. Chrom-ART)
- Versteht Rechtecke (X/Y), nicht nur lineare Bytes
- Zeilenoffsets am Ende jeder Zeile — Ausschnitte aus Framebuffer-Layouts

Notizen

Für GUIs: Button aus Hintergrundbild kopieren, ohne CPU-Pixelschleifen.

Scatter-Gather (verkettete Listen)

- Viele kleine Puffer im RAM verstreut
- Neu-Konfigurieren nach jedem Puffer kostet CPU/IRQs
- Scatter-Gather: Deskriptor-Liste (SRC, DST, COUNT, Next) im RAM
- DMA lädt den nächsten Auftrag selbst nach

Notizen

Königsklasse: CPU schreibt die Auftragskette; DMA arbeitet sie autonom ab.

Grenzen von DMA

- Verschiebt Bytes — keine allgemeine Logik
- Kein `if` auf Nutzdaten, keine Berechnung pro Byte
- Kein Protokollverständnis (z. B. I2C-ACK): nur Reaktion auf DREQ/Handshake
- Für Entscheidungen bleibt die CPU zuständig

Notizen

Assistent ohne Gehirn: transportiert zuverlässig; bewertet und filtert nicht.

Aufgabe — CS und Audiofenster

Darf CS bei DMA-TC bereits HIGH werden? Was folgt aus 30 ms Nachfüllzeit im Audio-Beispiel?

Notizen

DMA-TC garantiert nicht den Leitungsabschluss. CS folgt dem Geräte- und Controllertiming. 30 ms liegen über den idealen 23,22 ms je Hälfte: Underrun beziehungsweise Overrun. Siehe Lebenszyklus und Ping-Pong.

Ausblick Labor: DMA in Renode

Rechenmodell, keine Renode-Messung. 4096 Byte, 115200 Baud, 8N1.

Ideale Leitungszeit: $4096 \times 10 / 115200 = 355,56 \text{ ms}$.



Heartbeat schaltet die LED in main(), etwa alle 10 ms nach Zeitvergleich.

Ein Timer darf die Zeitbasis sein. Der Wechsel darf nicht autonom in Hardware oder Timer-ISR liegen.

Vergleich nur bei identischen Daten und UART-Einstellungen.

Getrennt protokollieren: korrekter Bytestrom, Main-Fortschritt, Heartbeat-Luecken, DMA-TC, Leitung fertig.

Performance in Mips ist keine MCU-Taktfrequenz. Hostlaufzeit ist keine Transferzeit.

Ohne Baud- und Handshake-Modell bleibt der Lauf ein Funktionstest, keine Entlastungsmessung.

Im Repository kein DMA-Modell und keine .repl. Diese Adressen bleiben fiktiv.

Abbildung 12: 4096 Byte, 8N1, 115200 Baud: ideal etwa 355,56 ms. Das ist eine Rechnung, keine Messung.

Geplantes Szenario (Details im Übungsblatt):

- Langer String (z. B. HTML-Header) über UART
- Runde 1: Polling — CPU blockiert sichtbar
- Runde 2: DMA-Kanal — CPU rechnet/blinkt weiter, Text läuft im Hintergrund

Notizen

Dieselben Dämonen wie in der Theorie: erst blockierend, dann asynchron entlastet.

Messung 1: Polling (blockierend)

- LED als Heartbeat der main-Schleife
- Klassische UART-Sendeschleife:

```

void send_uart_polling(const char *text, uint32_t len) {
    for (uint32_t i = 0; i < len; i++) {
        while (!(UART_STATUS_REG & TX_EMPTY)) { }
        UART_TX_REG = text[i];
    }
}
  
```

Notizen

Vergleichswert: die C-Schleife, die wir theoretisch als Engpass begründet haben.

Erwartung: Timer-/LED-Beweis

- Während langer Polling-Ausgabe: Text im UART-Monitor
- LED steht (nahezu) still — Heartbeat pausiert
- Interpretation: Polling blockiert Multitasking in der Hauptschleife

Notizen

Laboridee: Solange gesendet wird, kommt die `main`-Schleife nicht zum Blinken.

Messung 2: Umstieg auf DMA

- Polling-Schleife ersetzen durch MMIO-Konfiguration des DMAC
- Generisches Rezept in vier Schritten (herstellerübergreifend)

Notizen

Registerlayout im Labor aus Datenblatt / Renode-Modell — Bits hier didaktisch.

Schritt 1: UART für DMA scharfschalten

- Standard: UART erwartet CPU-Writes
- Enable-Bit für DMA-Request (DREQ), wenn TX-Register Platz hat

```
#define UART_CR3      (*(volatile uint32_t *) (UART_BASE + 0x08u))
#define UART_DMA_EN  (1u << 7) /* Bitlage fiktiv */

UART_CR3 |= UART_DMA_EN;
```

Notizen

Ohne Request vom UART startet der DMA nicht — Peripherie muss „rufen“ dürfen.

Schritt 2: Kanal konfigurieren

- SRC = String im SRAM, DST = UART-TX-Register (fest)
- COUNT = Länge; Memory-Increment an, Peripherie-Increment aus

```
DMA_CH1->SRC_ADDR = (uint32_t)(uintptr_t)html_string; /* fiktiv *
DMA_CH1->DST_ADDR = (uint32_t)(uintptr_t)&UART_TX_REG;
DMA_CH1->COUNT   = string_laenge; /* Einheit laut Controller */
DMA_CH1->CONTROL   = DMA_MINC_EN | DMA_DIR_M2P; /* kein blindes |
```

Notizen

Exakt die Parameter aus dem Theorieteil — Array wandert, TX-Adresse bleibt starr.

Schritt 3: Interrupts aktivieren

- Transfer-Complete-Enable im DMA
- PLIC (bzw. Interrupt-Controller) für den Kanal freischalten

```
/* TCIE in demselben kontrollierten CONTROL-Wort, nicht per Altz
enable_interrupt(DMA_CH1_IRQn); /* Pseudocode */
```

Notizen

Ein IRQ am Ende statt eines IRQs pro Byte — Abschluss und ggf. nächster Auftrag.

Schritt 4: Enable (Startschuss)

- ENABLE-Bit setzen → Transfer startet hardwareseitig
- C-Funktion kehrt sofort zurück — CPU ist frei

```
void start_dma_transfer(void) {
    DMA_CH1->CONTROL = DMA_MINC_EN | DMA_DIR_M2P
        | DMA_TCIE_EN | DMA_ENABLE_BIT;
}
```

Notizen

Ein Bit, und der Hintergrundtransfer läuft — Triumph der Delegation.

Folie

72/75

Erwartung mit DMA

- Text erscheint weiter im Monitor
- Der Heartbeat kommt aus `main()`, nicht aus einer Timer-ISR und nicht aus Hardware-PWM
- Die LED kann weiterlaufen und trotzdem Busstalls haben. Das ist kein Beweis für null Last
- DMA-TC heißt: letztes Element übergeben. FIFO und Schieberegister können noch senden

Notizen

Zielbild im Simulator: System „lebt“, während Bytes asynchron ausgehen.

Folie

73/75

Interrupt-Handling am Ende

- `COUNT = 0` → Transfer Complete
- In der ISR Status quittieren (Flag clear), sonst erneutes Feuern

```
void dma_channell_isr(void) {  
    if (DMA_STATUS_REG & TRANSFER_COMPLETE) {  
        DMA_CLEAR_REG = CLEAR_TRANSFER_COMPLETE; /* W1C: schreib  
        /* optional: nächsten Transfer starten */  
    }  
}
```

Notizen

Wie beim SysTick: Grund lesen, Flag löschen, zurück — Mission erfüllt.

Zusammenfassung Block 6

- Polling blockiert; Interrupt-Stürme kosten Overhead
- DMA: spezialisierter Coprozessor für Speichertransfers
- DMAC als Bus-Master, Cycle Stealing mit dem Arbiter
- Kanal: SRC, DST, COUNT, Breite, Increment
- Cache: Richtung, Besitz und Plattformsequenz. PMP ist kein Cache-Schalter
- Circular Mode, HT/TC → Ringpuffer und Ping-Pong

Notizen

CPU bleibt Gehirn; DMA ist der Muskel für Massenbewegungen.

Ausblick und Abschluss

- Übungsblatt: Polling vs. DMA in Renode nachvollziehen
- Nächster Block: moderne CPU-Beschleunigung (u. a. Out-of-Order)

Vielen Dank — viel Erfolg im Labor.

Notizen

Quellen, keine Übernahme als RISC-V-API: RISC-V Privileged, PMA/PMP; CMO/Zicbom; Linux DMA API zu Cachelines; AN4839 nur als Cortex-M7-Beispiel; STM32F4-HAL nur als Beispiel getrennter DMA- und UART-Abschlüsse; Renode Time Framework. Adressen dieser Folie bleiben fiktiv. Kein DMA-Modell im Repository.

Wenn Terminal flüssig schreibt und die LED weiterblinkt, ist die Architektur verstanden. Bis zum nächsten Block!